NYC Taxi MDM

End-to-End Learning Guide

AWS Data Engineering, taught through one real deployed platform

Goal: understand every important part of this platform deeply enough to explain the architecture, implementation decisions, AWS services, Terraform, data flow, security, CI/CD and troubleshooting confidently in an interview.

Every factual claim here was verified against the repository or the live AWS account. Where something is not implemented, it is labelled. Where something is managed outside Terraform, it is labelled. Nothing is inferred and presented as fact.

Verification note — what changed from the earlier draft

This guide replaces an earlier draft. Before writing it, the repository and the live account were re-inspected. Four things were corrected or sharpened:

Claim	Verified position
Step Functions has 17 states	True, but misleading on its own. There are 6 functional stages; the other 11 states are poll loops, success/failure terminals and the notification path
sync-pipeline-runs is part of the pipeline	It is a Glue job but NOT a Step Functions stage — zero references in the state machine definition. It runs separately
RDS is private because publicly_accessible = false	Both are true but they are different claims. The RDS subnets route to an internet gateway — they are public subnets. Protection comes from having no public address and from the security group, not from subnet isolation
warehouse/ is part of the catalog	It is not crawled. Zero references in the crawler configuration. Only bronze, silver and gold are crawled

VERIFIED — Everything in this guide was checked against terraform/, pipeline/glue/jobs/, services/, .github/workflows/, the Step Functions definition, and live AWS API calls.

Part 1 — The project at a glance

1.1 The problem being solved

A dashboard reports that pickups in a zone fell 40% last quarter. Three explanations are possible, and only one is a business insight:

1. Fewer people actually took taxis there — a real signal.
2. The zone was renamed, so the report is splitting one zone into two.
3. The zone was re-classified into a different borough.

Cases 2 and 3 are data-management failures wearing the costume of an insight. Master Data Management is the discipline that keeps them distinguishable, and Slowly Changing Dimension Type 2 is the technique that preserves the history needed to tell them apart.

That is the problem this platform exists to solve. Everything else is the machinery.

1.2 The data

Two datasets with opposite characteristics, which is what makes the engineering interesting.

	Transactional	Reference / master
What	NYC yellow taxi trips	NYC TLC taxi zone lookup
Volume	2,851,125 rows	265 rows
Change rate	Append-only, reprocessed wholesale	Rare, but every change matters
If you lose a row	Reprocess and it returns	History is gone forever
Correct treatment	Cheap bulk storage, columnar, reprocessable	Versioned, transactional, audited
Lands in	S3 data lake	RDS PostgreSQL

1.3 Why each technology is here

Technology	Why it is in this platform
S3	Cheap, durable, effectively unlimited storage. The system of record for all trip data
AWS Glue	Serverless Spark. The pipeline runs on demand, so there is no cluster worth managing
Delta Lake	ACID writes, time travel and schema evolution over files in S3
RDS PostgreSQL	The one component that needs real transactions — the SCD2 compare-and-version step must be atomic
Redshift Serverless	Columnar MPP warehouse to serve the star schema without running a cluster continuously
QuickSight	BI layer. SPICE caching decouples dashboard speed from warehouse availability
Athena	SQL over the lake, and — more valuably — an independent engine for cross-engine reconciliation
Step Functions	Orchestration with declarative retries, error routing and durable execution history
Terraform	Reproducible, reviewable infrastructure. A plan diff is the change-control mechanism
GitHub Actions + OIDC	CI/CD with no long-lived AWS credential anywhere

1.4 The 30-second explanation

Say this: It is an AWS data platform that processes 2.85 million NYC taxi trips through a Bronze/Silver/Gold data lake on S3, masters the taxi-zone reference data in PostgreSQL with full SCD Type 2 history, and serves the result as a Redshift star schema behind a QuickSight dashboard. One Step Functions execution runs the whole chain, and everything is deployed by Terraform through GitHub Actions using OIDC — no AWS access keys exist.

1.5 The 2-minute interview explanation

Lead with the problem, not the service list.

4. **The problem** — two datasets needing opposite treatment — 2.85M disposable trip rows, and 265 reference rows where every historical change must survive.
5. **The lake** — trips land in S3 as Bronze, are cleaned into Silver as Delta, and aggregated into five Gold summaries. Each layer is reproducible from the one before it.
6. **The MDM side** — zones go into RDS PostgreSQL through a stored procedure implementing SCD Type 2 — insert, no-change, or expire-and-version.
7. **The convergence** — a warehouse export job reads Silver and the mastered zones together and writes a plain-Parquet star schema, because Redshift COPY cannot read Delta and Gold has no dimensional keys.
8. **Serving** — Redshift holds fact_trips plus three dimensions; QuickSight ingests into SPICE.
9. **The proof** — 18 in-warehouse reconciliation checks pass and 33 days tie out against Athena computing the same totals from Gold independently — to the cent.
10. **The engineering** — all Terraform, deployed via OIDC-federated GitHub Actions, with plan on pull requests and a manually gated apply.

1.6 The 5-minute architecture walkthrough

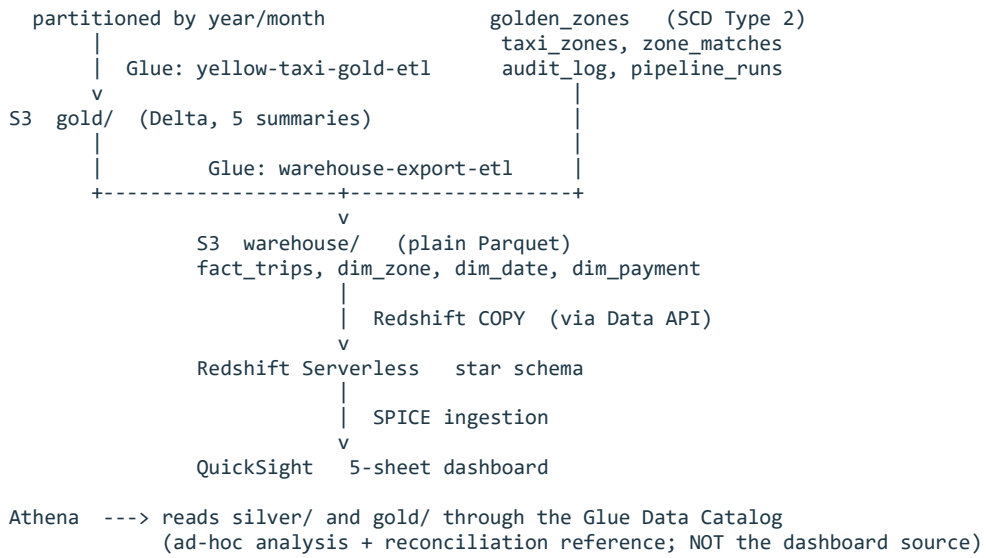

Figure 1 — the two pipelines and where they converge.

Walk it in that order and the design justifies itself: volume on the left, meaning on the right, a deliberate convergence point, then serving.

Part 2 — AWS fundamentals this project depends on

Each concept is defined, then immediately tied to where it appears here.

2.1 Accounts, regions, availability zones

- **Account** — the billing and isolation boundary. Resource names are unique within it, and every ARN contains its ID.
- **Region** — a geographic cluster of data centres. Services are regional — a bucket in one region is not visible to a service configured for another.
- **Availability Zone** — one or more physically separate data centres within a region, independently powered. Multi-AZ deployment survives one failing.

This platform runs entirely in one region. The Redshift workgroup requires subnets spanning at least three AZs, which is why the configuration keeps a separate subnet list for it — the RDS subnet list is not the same one.

> **Interview framing:** 'everything is in one region because it is a single-region analytical workload with no cross-region consumers. Multi-region would add cost and replication complexity to solve a problem this platform does not have.'

2.2 ARNs

An Amazon Resource Name uniquely identifies any resource:

```
arn : partition : service : region : account-id : resource
```

ARNs are how IAM policies name what they permit. Scoping a policy to one exact ARN rather than a wildcard is the difference between least privilege and a shrug. In this platform the Glue role's secret permission names one secret ARN, not all secrets.

2.3 IAM — users, roles, policies, trust policies

Concept	What it is	In this project
User	A long-lived identity with credentials	Used only by the human operator; no service uses one
Role	An identity assumed temporarily, no password	Every service and CI workflow uses one
Identity policy	What the identity may do	Per-service, scoped to named resources
Trust policy	Who may assume the role at all	The security boundary for CI — pins repository and context
AssumeRole	The act of taking on a role	Called by Glue, Step Functions and GitHub Actions
STS	The service issuing temporary credentials	Issues short-lived keys for every assumption

The two-question model is worth internalising, because they are answered by different documents: the trust policy answers WHO CAN BECOME THIS, and the identity policy answers WHAT CAN IT THEN DO. Nearly every confusing IAM failure is one of those two mixed up with the other.

2.4 OIDC federation and temporary credentials

OpenID Connect lets AWS trust an external identity provider. GitHub signs a short-lived token describing exactly which repository, workflow and context is running; AWS validates the signature and exchanges it for temporary credentials.

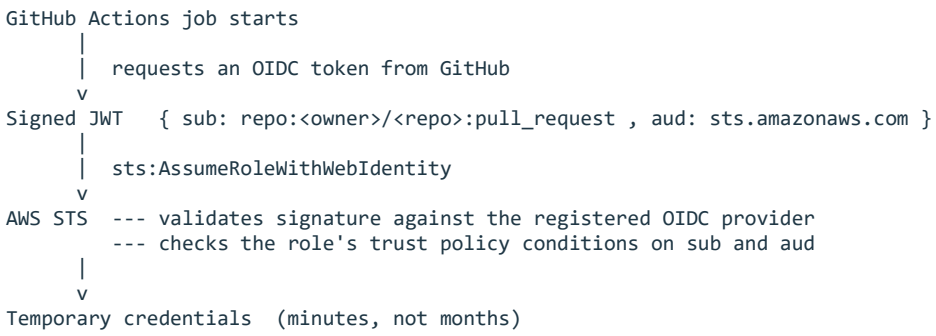

Figure 2 — how GitHub authenticates to AWS with no stored secret.

Why it matters here: no long-lived AWS access key exists anywhere in this project's automation. There is no key to leak, rotate or find in a settings page.

2.5 VPC, subnets, route tables, gateways

A VPC is a private network. A subnet is a slice of it bound to one AZ. The distinction between public and private is not a setting called 'public' — it is entirely determined by the subnet's route table.

Component	What it does
Route table	Decides where traffic for a destination goes. THIS is what makes a subnet public or private
Internet Gateway	Allows two-way internet traffic. A subnet routing to one is a public subnet
NAT Gateway	Outbound-only internet for private subnets. Nothing can initiate a connection inward
Elastic IP	The static public address the NAT gateway uses
Security group	A stateful firewall on a resource. Return traffic is automatic
Gateway endpoint	A route to S3 that never leaves AWS. No hourly charge, no data charge
Interface endpoint	A private IP for a service inside your subnet. Billed hourly

TRAP — A security group can reference another security group instead of an IP range. This platform's database group admits the Glue group, not a CIDR — which keeps working when serverless workers get new addresses on every run, as they do.

2.6 KMS keys — AWS-managed versus customer-managed

	AWS-managed	Customer-managed (CMK)
Created by	AWS, automatically, per service	You
Key policy	Not editable	Yours to write
Rotation	Automatic, not configurable	Configurable
Cost	No monthly charge	Monthly charge per key
Cross-account	Not possible	Possible

This distinction produces a real limitation in this platform — see Part 15.

2.7 Secrets Manager, CloudWatch, SNS, EventBridge

- **Secrets Manager** — stores credentials and returns them to authorised callers at runtime. The identifier is not sensitive; the value is.
- **CloudWatch** — metrics, logs, alarms and dashboards — the default telemetry destination.
- **SNS** — publish/subscribe topics that fan messages out to subscribers.
- **EventBridge** — an event bus. AWS services emit events; rules match them by pattern and route them to targets.

TRAP — CloudWatch alarms and EventBridge rules are two INDEPENDENT paths to SNS in this platform. Alarms do not flow through EventBridge. Confusing the two is a common interview stumble — see Part 13.

Part 3 — The complete architecture, by plane

The same system, sorted by the job each component does rather than by data flow. This is the view that makes an architecture diagram explainable.

Plane	Components	Responsibility
Data plane	S3 (bronze, silver, gold, warehouse, master, demo), Glue jobs, RDS, Redshift	Actually moves and stores data
Orchestration / control plane	Step Functions, EventBridge	Decides what runs, when, and what happens on failure
Metadata / catalog	Glue Data Catalog, 3 Glue crawlers, Athena	Describes what the files in S3 mean so SQL engines can read them
Security	IAM roles and policies, KMS CMK, Secrets Manager, VPC, security groups, NAT, S3 gateway endpoint, GitHub OIDC provider	Controls who and what may reach anything
Observability	CloudWatch logs/metrics/alarms/dashboard, SNS topic, EventBridge rule, pipeline_runs table	Makes failure visible and reaches a human
CI/CD	Terraform, S3 remote state, 3 GitHub Actions workflows, 2 OIDC roles	Turns a reviewed code change into deployed infrastructure
BI / presentation	QuickSight VPC connection, data source, SPICE dataset, analysis, dashboard	Serves the result to a human

MANUAL / OUTSIDE TERRAFORM — The entire BI/presentation plane — all six QuickSight resources — is created through the API and is NOT managed by Terraform. This is the only plane where a rebuild from code alone would not reproduce the system. Covered in Part 21.

Part 4 — Amazon S3, the data lake

4.1 Concepts

Concept	What it means	Why it matters here
Bucket	Top-level container, region-bound	One for the lake, one for Terraform state
Key	The object's full name	Slashes are just characters — there are no real folders
Prefix	A leading substring used to group objects	How the medallion layers are separated
Versioning	Keeps every version of an object	Makes an overwrite or delete recoverable
Lifecycle rule	Transitions or expires objects by age	Controls long-term storage cost
Default encryption	SSE applied to every write	Set to the customer-managed key on the lake
Strong consistency	A read after write returns the new object	No stale-read handling needed in job code

4.2 Why S3 rather than a database for the lake

Three reasons, in order of importance:

11. **Cost at volume** — storing millions of rows as columnar files is an order of magnitude cheaper than the same data in a database's storage engine.
12. **Schema on read** — you decide the structure when reading, so Bronze can accept whatever the source publishes without a migration.
13. **Engine independence** — the same files are readable by Spark, Athena and Redshift. Data in a database is reachable only through that database.

4.3 The actual prefixes in this project

Prefix	Contents	Format	Crawled?
bronze/	Raw trip files and the zone reference CSV	Parquet, CSV	Yes
silver/	Cleaned trips, partitioned by pickup year/month	Delta	Yes (delta target)
gold/	Five business summaries	Delta	Yes
warehouse/	Star-schema snapshot for loading	Plain Snappy Parquet	NO — see 4.4
master/	Published mastered zone reference	Parquet	No
demo/	Delta demonstration and its evidence file	Delta, JSON	No
glue/scripts/	Job scripts, uploaded by Terraform	Python	No
glue/temp/	Spark temporary directory	Internal	No
athena-results/, logs/, quality/, metadata/, checkpoints/	Supporting artifacts	Various	No

A separate bucket holds Terraform state: private, versioned, encrypted with S3-managed encryption, with conditional-write locking.

4.4 Why warehouse/ is deliberately not crawled

VERIFIED — Confirmed by inspection: the crawler configuration contains zero references to the warehouse prefix. Only bronze, silver and gold are crawled.

The catalog exists so that SQL engines can discover schema. Nothing queries the warehouse prefix through the catalog:

- Redshift reads it with COPY, which is told the schema by the target table — it does not consult the Glue Data Catalog.
- Athena queries silver and gold, not the warehouse snapshot.
- QuickSight reads Redshift, not S3.

Crawling it would create catalog tables nothing queries, add a scheduled cost, and invite someone to query a staging artifact as though it were a curated dataset.

The general principle: catalog what is meant to be queried by SQL engines. A staging area consumed by exactly one loader with a known schema is not that.

4.5 Why versioning matters here

Two distinct benefits:

- **On the state bucket** — a corrupted or truncated Terraform state file can be rolled back to the previous version. Without versioning, a bad write is unrecoverable and you rebuild state by hand.
- **On the data lake** — an accidental overwrite of a script or a data file is recoverable.

TRAP — Versioning also means deletes are soft. Storage keeps growing until a lifecycle rule expires non-current versions, which is a cost surprise people discover months later.

Interview questions on this

Q Why S3 instead of a database for the data lake?

Cost at volume, schema-on-read so Bronze accepts whatever the source publishes, and engine independence — Spark, Athena and Redshift all read the same files. A database would make the data reachable only through that database, and would cost far more per GB.

Q Why is the warehouse prefix not crawled?

Nothing queries it through the catalog. Redshift COPY is told the schema by the target table; Athena queries silver and gold; QuickSight reads Redshift. Crawling it would produce catalog tables nothing uses.

Q What does S3 strong consistency change for pipeline design?

A read immediately after a write returns the new object, so job code needs no retry loop for eventual consistency. That used to be real design work before it became strongly consistent.

Part 5 — AWS Glue in depth

5.1 Four different things share the name

Component	What it does	Used here?
Glue job (Spark)	Serverless Spark — supply a script, AWS provisions and tears down workers	Yes, 5 jobs
Glue job (Python Shell)	A single lightweight Python process, no Spark	Yes, 1 job

Glue Data Catalog	Metadata store of table definitions	Yes, 4 databases
Glue crawler	Scans S3, infers schema, registers catalog tables	Yes, 3 crawlers
Glue connection	Network configuration placing a job inside a VPC	Yes, 1 NETWORK connection
Glue bookmarks	State tracking so a job processes only new data	NOT used — see 5.5

5.2 ETL vs crawler vs catalog vs connection

These are constantly confused. The clean separation:

```
Glue ETL job        MOVES AND TRANSFORMS DATA
                    reads files -> computes -> writes files or database rows

Glue crawler        DISCOVERS SCHEMA
                    reads a sample of files -> writes table definitions to the catalog
                    moves no data

Glue Data Catalog   STORES METADATA
                    table names, columns, types, partitions, file locations
                    holds no data itself

Glue connection     PROVIDES NETWORK PLACEMENT
                    puts a job's workers on ENIs inside a VPC subnet
                    moves no data, stores no metadata
```

Figure 3 — four Glue components, four separate jobs.

5.3 Spark concepts you need

- **Driver and executors** — the driver plans; executors do the work in parallel across partitions.
- **Partition** — a chunk of data one task processes. Parallelism is bounded by partition count.
- **Shuffle** — redistribution of data across executors, triggered by joins and wide aggregations. The expensive operation.
- **Lazy evaluation** — transformations build a plan; nothing executes until an action.
- **DataFrame** — the standard Spark abstraction — typed columns, SQL-like operations.
- **DynamicFrame** — a Glue-specific wrapper that tolerates inconsistent schemas and can resolve type ambiguity. Converts to and from a DataFrame.

This project uses DataFrames and the Delta DataSource API rather than DynamicFrames. The source schema is known and stable, so the schema-flexibility DynamicFrames provide would buy nothing while making the code less portable to plain Spark.

5.4 Glue configuration in this project

VERIFIED — Live inspection of all six jobs. Worker type and VPC attachment as shown.

Job	Type	Workers	In VPC	Purpose
yellow-taxi-silver-etl	Spark	G.1X x2	no	Bronze Parquet to Silver Delta: clean, validate, type, partition
yellow-taxi-gold-etl	Spark	G.1X x2	no	Silver Delta to five Gold summaries
golden-zone-etl	Spark	G.1X x2	YES	Zone CSV into golden_zones via the SCD2 stored procedure
warehouse-export-etl	Spark	G.1X x2	YES	Silver + mastered zones to warehouse Parquet
sync-pipeline-runs	Python Shell	n/a	YES	Glue run history into the pipeline_runs table
delta-demo	Spark	G.1X x2	no	Delta time-travel demonstration (non-production)

The pattern is exact: a job is in the VPC if and only if it talks to the database. Three do; three do not.

Inputs, outputs and dependencies

Job	Reads	Writes
silver-etl	bronze/transactions/	silver/ (Delta, partitioned)
gold-etl	silver/	gold/ (5 Delta tables)
golden-zone-etl	bronze/reference/ zone CSV	RDS: golden_zones via stored procedure
warehouse-export-etl	silver/ AND RDS mastered zones	warehouse/ (4 plain Parquet datasets)
sync-pipeline-runs	Glue job run history API	RDS: pipeline_runs
delta-demo	bronze/reference/ zone CSV	demo/ only

5.5 Why bookmarks are not used

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — Glue job bookmarks are not enabled. The pipeline reprocesses wholesale on every run.

This is deliberate. Bookmarks let a job process only new data, which is valuable for high-frequency incremental loads. Here, full reprocessing means every layer is reproducible from the one before it, and the reconciliation is meaningful because fact_trips is a lossless projection of the entire Silver table. Incremental processing would make row-count reconciliation far harder to reason about for a dataset this size.

5.6 Scripts, arguments and IAM

Job scripts live in the repository, are uploaded to S3 by Terraform, and are referenced by the job definitions. The deployed script is therefore always the reviewed one; there is no console editing.

All Glue jobs assume a shared Glue execution role, which grants: read its own secret ARN, read and write the relevant lake prefixes, write to its CloudWatch log group, and — for VPC jobs — manage network interfaces.

TRAP — Glue JOB ARGUMENTS are not SPARK CONFIGURATION. Two Spark SQL settings were passed as ordinary job arguments and silently reached nothing. Jobs using only the DataFrame API were unaffected; the demo job needed the Delta configuration passed explicitly through --conf. If a Spark setting appears to have no effect, check which mechanism delivered it.

Interview questions on this

Q Why Glue rather than EMR?

The pipeline is batch and runs on demand. EMR means managing a cluster lifecycle for a job that runs for minutes. Glue provisions, runs and tears down with no idle cost and no cluster to operate.

Q DataFrames or DynamicFrames, and why?

DataFrames. DynamicFrames exist to tolerate inconsistent or ambiguous source schemas. This source schema is known and stable, so they would add Glue-specific coupling for no benefit.

Q Why are only some jobs in the VPC?

A job joins the VPC if and only if it reaches the private database. Putting the others there would add ENI setup, a NAT dependency and slower starts for nothing.

Part 6 — Bronze, Silver, Gold

6.1 The medallion idea

Progressive refinement, with each layer reproducible from the one before it. The value is diagnostic: when a number is wrong, you trace backwards through named layers instead of through an undifferentiated pile of jobs.

Layer	Principle	In this project
Bronze	Raw, unmodified, replayable. Never edit it	Trip files and zone CSV exactly as published
Silver	Cleaned, validated, typed, deduplicated where appropriate	2,851,125 rows in Delta, partitioned by pickup year and month
Gold	Business-ready aggregations	Five Delta summaries: daily, vendor, borough, payment, hourly

Gold table	Rows
daily_summary	33
vendor_summary	95
borough_summary	203,611
payment_summary	128
hourly_summary	748

6.2 Why preserve Bronze at all

Because every downstream assumption will eventually be wrong. If a cleaning rule turns out to have dropped valid rows, Bronze is what lets you fix the rule and reprocess. Without it, the error is permanent and the only remedy is re-acquiring the source.

6.3 Where Delta is and is not used

Location	Format	Why
bronze/	Parquet and CSV as published	Raw means raw. Converting it would make it not-raw
silver/	Delta	ACID writes and schema evolution on the main working table
gold/	Delta	Same, plus consistent reads while aggregations are rewritten
warehouse/	PLAIN Parquet, not Delta	Redshift COPY cannot read Delta — this is the reason the layer exists
demo/	Delta	Non-production demonstration only

This single table answers a very common interview question: 'you used Delta — why is the warehouse export plain Parquet?' Because COPY cannot read a Delta transaction log, and Gold is pre-aggregated with no dimensional keys so it cannot supply a trip-grain fact table.

Part 7 — Delta Lake

7.1 What Parquet alone does not give you

Parquet is a columnar file format. It is efficient and self-describing, and it has no notion of a table, a transaction or a version. Consequences:

- A job failing mid-write leaves partial files that readers will happily read.
- Two writers can interleave and corrupt a logical table.
- There is no way to ask what the data looked like before the last write.
- Adding a column means rewriting every file.

7.2 What Delta adds

Delta keeps a transaction log — a directory of ordered JSON commits — alongside the Parquet files. The log, not the file listing, defines what the table contains.

```
silver/
  _delta_log/
     00000000000000000000.json     <- commit 0: which files are IN the table
     00000000000000000001.json     <- commit 1: files added / removed
     00000000000000000002.json     <- commit 2: ...
  pickup_year=2025/pickup_month=1/part-0000....snappy.parquet
  pickup_year=2025/pickup_month=1/part-0001....snappy.parquet

A reader consults the log, not the directory listing.
Files not referenced by the log are invisible -> partial writes cannot be read.
```

Figure 4 — the transaction log is what makes files a table.

Property	How the log provides it
Atomicity	A write is visible only when its commit lands. Partial files are unreferenced
Time travel	Each commit is a version. Reading version N replays the log to that point
Schema evolution	The commit records the schema; older files read NULL for new columns
Concurrency	Commits are ordered, so conflicting writers are detected

7.3 The demonstration in this project

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — The Delta demonstration is NON-PRODUCTION. It writes only to the demo/ prefix, reads only the Bronze zone CSV, and touches no production table. It is not a Step Functions stage and is not part of any production data flow.

Version	Operation	Result
v0	Initial write	265 zones, 4 columns
v1	Update	One row's service_zone rewritten in place
v2	Write with mergeSchema	3 rows appended carrying a new zone_category column

Read back with no rewriting:

- version 0 returns the original 4-column schema AND the pre-update value
- version 1 shows the changed value with the original schema
- the current version has 5 columns, with the 265 pre-existing rows reading NULL in the new one
- DESCRIBE HISTORY lists all three versions with their operations

The table is rebuilt on every run, so version numbers are deterministically 0, 1 and 2. Evidence is written to a JSON file under the demo prefix.

What it proves: that time travel returns both the old schema and the old values, and that schema evolution does not require rewriting history. Those are the two claims people make about Delta without ever demonstrating them.

Part 8 — Master Data Management and SCD Type 2

8.1 Why master data is different

	Transactional data	Master data
Volume	Millions of rows	Hundreds of rows
Change	Append-only	Updates in place, rarely
Value per row	Low individually	Very high — everything references it
If wrong	One bad trip record	Every report built on it is wrong
History	The event IS the history	Must be explicitly preserved

8.2 Core MDM vocabulary

Term	Meaning	In this project
Golden record	The single authoritative version of an entity	A row in golden_zones with is_current true
Business key	The natural, stable identifier	location_id — used for all current-record resolution
Surrogate key	A system-generated row identifier	Present on golden_zones rows, one per VERSION
Duplicate detection	Finding records that describe the same real entity	Not needed — the source has one row per location_id
Fuzzy matching	Probabilistic matching on non-identical strings	NOT implemented — see below
Survivorship	Rules deciding which value wins when sources conflict	Trivial here — a single authoritative source

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — Fuzzy matching and multi-source survivorship are NOT implemented. There is exactly one authoritative source for taxi zones — the TLC lookup file — with one row per location_id. Building probabilistic matching against a single clean source would be machinery with nothing to do. zone_matches records a validated deterministic mapping, not a probabilistic one.

Be ready to say this out loud: 'I did not implement fuzzy matching because the source is unambiguous. Adding it would have demonstrated a technique while solving no problem this dataset has.' Interviewers respect scoping far more than unused machinery.

8.3 SCD Type 2, precisely

Type 1 overwrites and loses history. Type 2 closes the current row and inserts a new version. That is what makes 'what was this zone called when that trip happened' answerable.

```
Incoming zone row
      |
      |  standardise, compute record_hash
      v
sp_upsert_golden_zone(location_id, attributes, record_hash)
```

```
|
+--- no current row exists ----------------> INSERTED
|                                            version 1, is_current = true
|
+--- current row, hash MATCHES -------------> NO_CHANGE
|                                            write nothing at all
|
+--- current row, hash DIFFERS -------------> UPDATED
                                             close old row (end_date, is_current=false)
                                             insert new version, is_current = true
```

Figure 5 — the three transitions, all proven against the live database.

Getting NO_CHANGE right is what separates a working implementation from one that silently creates a new version on every run and doubles the table weekly.

Why a stored procedure and not Spark: the compare-and-version step must be atomic. Read current, decide, close it, insert the new one — as one transaction, or two concurrent runs both conclude they are creating version 2. A data lake cannot provide that; PostgreSQL can.

8.4 The tables

Table	Role
taxi_zones	Source-aligned reference records, one row per location_id
golden_zones	Mastered records with full SCD Type 2 version history — MORE rows than zones, by design
zone_matches	Validated mapping from a source zone to its golden record
audit_log	Row-level change history written by an AFTER INSERT OR UPDATE trigger
pipeline_runs	Operational run tracking, mirrored from Glue job run history

8.5 Current-record resolution — and the bug worth telling

Downstream consumers need the current version of each zone. There are two ways to get it and one is wrong in a way that is easy to miss.

zone_matches carries a pointer to a specific golden version row. Joining on that pointer looks natural and works perfectly — until a zone is updated. Nothing repoints it when a new version supersedes the old one, so the join silently drops every zone that has ever changed.

It surfaced as a dimension table one row short: 264 where 265 was required.

```
-- WRONG: points at one specific version row, never repointed
JOIN golden_zones g ON g.golden_zone_row_id = zm.golden_zone_row_id

-- RIGHT: resolves by business key, always finds whichever version is current
FROM zone_matches zm
JOIN taxi_zones   t ON t.location_id = zm.source_zone_id
JOIN golden_zones g ON g.location_id = t.location_id
WHERE g.is_current
```

Going through taxi_zones also excludes a test fixture structurally rather than by filtering on a magic number — which matters, because that fixture has a current version of its own and would otherwise inflate the count.

The lesson to state in an interview: a count that is almost right is more dangerous than one that is obviously broken. The fix was to change the resolution path, not to adjust a threshold until the test passed.

8.6 Vendors — historical and frozen

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — Vendor mastering is NOT active. vendor_etl.py, sp_upsert_vendor.sql and test_vendor.sql exist in the repository and the vendors table and its procedure still exist

in the database because the frozen migrations create them — but nothing references them and no pipeline stage runs them. They are retained as the origin record of an earlier iteration, not as active architecture.

There is also a principled reason not to revive them. No authoritative vendor master exists for this dataset, and the data contains a vendor code that no available source can name. Creating a vendor dimension would mean inventing master data inside a master-data project — the precise failure MDM exists to prevent. vendorid is carried as a degenerate dimension on the fact table instead.

Interview questions on this

Q What is SCD Type 2 and how did you implement it?

Changes never overwrite: the current row is closed with an end date and a new version is inserted. I implemented it as a PostgreSQL stored procedure comparing a record hash, with three paths — INSERTED, NO_CHANGE, UPDATED. It is a procedure because the compare-and-version step must be atomic.

Q How do you detect duplicate taxi zones?

I do not need to. The TLC lookup has exactly one row per location_id, so there is no duplicate problem to solve. zone_matches records a validated deterministic mapping. Fuzzy matching would be machinery with nothing to do.

Q Why is vendor data not actively mastered?

No authoritative vendor source exists and the data contains a vendor code nothing can name. Mastering it would mean inventing master data. It is a degenerate dimension on the fact table instead. The old vendor artifacts are frozen and referenced by nothing.

Part 9 — RDS PostgreSQL

9.1 RDS versus self-hosting versus Aurora

Option	You manage	AWS manages
PostgreSQL on EC2	OS, patching, backups, failover, storage growth, monitoring	The virtual machine
RDS PostgreSQL	Schema and queries	Engine, patching, backups, failover, storage
Aurora PostgreSQL	Schema and queries	All of the above plus a distributed storage layer

VERIFIED — This project uses RDS PostgreSQL on a single db.t3.micro instance. It is NOT Aurora.

Why not Aurora: a few hundred reference rows do not need a distributed storage engine. A single small instance is sufficient and materially cheaper. Choosing the larger service without a workload that justifies it is a cost, not a credential — and an interviewer will ask.

9.2 Concepts

Concept	Meaning	Setting here
Instance class	Machine size	db.t3.micro
Subnet group	Which subnets the instance may live in	2 subnets, 2 AZs
publicly_accessible	Whether it gets a public address at all	false
Parameter group	Engine configuration	Enforces SSL
Security group	Stateful firewall	Admits only the Glue security group on 5432

Automated backups	Point-in-time recovery window	7 days
Deletion protection	Blocks accidental deletion	Enabled
Multi-AZ	Synchronous standby in another AZ, automatic failover	NOT enabled — single instance

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — Multi-AZ is not enabled. This is a training and portfolio platform; the master data is rebuildable from the Bronze CSV by re-running the golden-zone job, and automated backups cover the history. Multi-AZ would roughly double the instance cost to protect against an AZ failure whose blast radius here is a re-run.

9.3 How the credential flows

```
Terraform generates a random password at deploy time
      |            (never in tfvars, never in source, never in an output)
      v
Secrets Manager secret   <-- encrypted with the customer-managed KMS key
      |
      |  the Glue job receives ONLY the secret ARN as a job argument
      v
Glue job at runtime: secretsmanager:GetSecretValue on that one ARN
      |
      v
psycopg2 connection over TLS to the private RDS endpoint
```

Figure 6 — no password ever appears in configuration or source.

Interview questions on this

Q Why RDS and not just put the master data in Redshift or S3?

The SCD2 compare-and-version step must be atomic. Redshift is an analytical warehouse without the transactional semantics for a safe read-decide-write, and S3 has no transactions at all. RDS is the only component here that can guarantee two concurrent runs do not both create version 2.

Q Is it Aurora?

No — RDS PostgreSQL, single db.t3.micro instance. Aurora's distributed storage solves a scale and availability problem a few hundred reference rows do not have.

Part 10 — Amazon Redshift

10.1 OLTP versus OLAP

	OLTP (RDS here)	OLAP (Redshift here)
Query shape	Read/write a few rows by key	Scan millions, aggregate a few columns
Storage	Row-oriented	Column-oriented
Optimised for	Transaction throughput and consistency	Scan and aggregation throughput
Indexes	Central to performance	No traditional indexes; sort keys and zone maps
Concurrency	Many small concurrent transactions	Fewer, much larger queries
In this platform	Master data, SCD2 versioning	fact_trips and three dimensions

Columnar storage is why a query touching 3 of 20 columns reads roughly 15% of the data. Massively parallel processing is why that scan is split across slices and executed at once.

10.2 Distribution and sorting

Concept	What it does
DISTSTYLE ALL	Full copy of the table on every node. Joins to it need no data movement
DISTSTYLE KEY	Rows placed by hash of a column. Co-locates matching rows for one join
DISTSTYLE EVEN	Round-robin placement. No skew, no join co-location
Sort key	Physical ordering, enabling block skipping at scan time
Compression / encoding	Per-column encoding, chosen automatically on load

10.3 This project's warehouse design

Table	Grain	Distribution	Rows
fact_trips	One row per taxi trip	EVEN	2,851,125
dim_zone	One row per taxi zone	ALL	265
dim_date	One row per calendar day	ALL	33
dim_payment	One row per TLC payment code	ALL	7

Why DISTSTYLE EVEN on the fact table

Three reasons, and being able to give all three is what makes this answer strong:

14. **A DISTKEY would buy nothing** — every dimension is DISTSTYLE ALL and therefore already present on every node. There is no redistribution left to avoid.
15. **It could only serve one join** — the fact table joins dim_zone twice — pickup zone and dropoff zone. A distribution key can co-locate at most one of them.
16. **It would introduce real skew** — NYC pickups concentrate heavily in a handful of zones, so hashing on zone would load a few slices far more than the rest.

10.4 The load

VERIFIED — The load file contains exactly 13 executable statements: 4 TRUNCATE, 4 COPY, 4 ANALYZE and 1 SELECT.

Terraform reads that file at plan time and passes its statements to the Redshift Data API through Step Functions, so the SQL file remains the single source of truth for the load. Restating the statements inside the state machine would create two places to change and one to forget.

TRAP — The load is NOT atomic. Redshift's TRUNCATE commits implicitly, so a failure part-way through leaves the warehouse partially refreshed rather than rolling back. What it is: a recoverable partial state. Why it exists: TRUNCATE is the simplest way to make a full reload idempotent. Why it was chosen: the pipeline is a full refresh with a single writer and re-running the same file recovers completely. What would change it: loading into staging tables and swapping them in a single transaction.

TRAP — ANALYZE cannot be granted. Table privileges are grantable; analysing a table is reserved to its owner. An automated load that analyses after loading must run AS the owning identity, not merely as a user holding full table permissions.

10.5 Why the warehouse is downstream of the lake, not instead of it

Loading raw trip data straight into Redshift would work and would be worse:

- Warehouse storage costs more per GB than object storage.

- Reprocessing means reloading rather than re-running a job over files already present.
- Only Redshift could read the data — Athena and Spark could not.
- Bronze would no longer exist as an independent replay source.

The lake is the system of record; the warehouse is a serving projection of it.

Interview questions on this

Q Why not put everything in Redshift?

Warehouse storage is expensive per GB, reprocessing becomes reloading, and the data becomes readable only by Redshift. Keeping the lake as the system of record preserves replay and lets Athena and Spark read the same files. Redshift holds only the serving projection.

Q Why DISTSTYLE EVEN?

The dimensions are DISTSTYLE ALL so they are already on every node — a distribution key has nothing left to co-locate. It could serve only one of the two zone joins, and hashing on zone would skew badly because pickups concentrate in a few zones.

Q How do you know the load is correct?

18 in-warehouse checks plus 33 days of counts and revenue tied out against Athena computing the same totals from Gold independently. fact_trips is a lossless projection of Silver, so the row counts must match exactly.

Part 11 — Crawlers, Data Catalog and Athena

11.1 The metadata chain

```
S3 files              (bytes, no schema anyone can query)
   |
   |  Glue crawler: samples files, infers columns and types, detects partitions
   v
Glue Data Catalog     (database -> table -> columns, types, partitions, location)
   |
   |  Athena reads table definitions from the catalog
   v
SELECT ... FROM silver_db.yellow_taxi WHERE pickup_year = 2025
```

Figure 7 — how files become queryable SQL tables.

VERIFIED — Three crawlers exist: bronze (S3 target), silver (Delta target), gold. Four catalog databases: bronze_db, silver_db, gold_db, master_db.

The Silver crawler uses a Delta target rather than a plain S3 target, so it registers the Delta table correctly rather than cataloguing the underlying Parquet files and the transaction log as if they were ordinary data.

11.2 Athena's actual role — and what it is not

Athena has two jobs here:

- **Ad-hoc analysis** — querying Silver and Gold without provisioning anything.
- **Reconciliation reference** — computing totals from Gold by a completely independent path from the warehouse. When two unrelated engines agree to the cent across 33 days, that is evidence. One engine agreeing with itself is not.

Architecture distinction people get wrong: Athena is NOT the source for QuickSight. QuickSight reads REDSHIFT over a VPC connection. Athena sits in a parallel branch off the lake and is not in the dashboard's serving path at all.

Interview questions on this

Q If QuickSight reads Redshift, why do you need Athena?

Independent verification. Athena computes the same totals from the Gold Delta tables by a different engine and a different code path. Agreement between them is meaningful evidence the warehouse load is correct. It is also the ad-hoc analysis route.

Q Why crawlers instead of writing table definitions by hand?

The crawler keeps partitions current as new ones appear. Hand-maintained definitions drift the moment the data changes shape and nobody notices until a query returns nothing.

Part 12 — Step Functions orchestration

12.1 Concepts

Concept	Meaning
State machine	The whole workflow definition
State	One step — Task, Choice, Wait, Succeed or Fail
Task	A state that calls another AWS service
Synchronous integration	The state waits for the invoked job to finish, not just to start
Retry	Declarative re-attempts with backoff, configured per state
Catch	Declarative error routing, configured per state
Execution history	A durable, inspectable record of every transition

12.2 The actual state machine

VERIFIED — 17 states total, StartAt = SilverETL. Six of them are functional stages; the other eleven are poll loops, terminals and the notification path. Four Glue jobs are invoked.

Figure 8 — six functional stages; the rest is poll control and failure routing.

12.3 Why sequential and not parallel

Not a limitation — the dependencies are real. WarehouseExport reads both the Silver Delta table and the mastered zones in RDS, so it cannot start before both upstream jobs finish. The Redshift COPY reads the warehouse snapshot. The SPICE refresh reads Redshift.

The only pair that could theoretically run concurrently is GoldETL and GoldenZoneETL — different sources, different targets. They are sequential because the marginal wall-clock saving does not justify the added failure-mode complexity for a pipeline that runs on demand.

12.4 What is NOT in the state machine

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — sync-pipeline-runs is NOT a Step Functions stage. Verified: zero references to it in the state machine definition. It is a Glue Python Shell job that mirrors Glue run history into the pipeline_runs table, and it is invoked separately — not as part of the orchestrated pipeline.

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — delta-demo is NOT a Step Functions stage either. It is a standalone demonstration job.

Why this matters in an interview: if you describe the pipeline as 'Step Functions runs all six Glue jobs', a careful interviewer who has read your repository will catch it. Four jobs are orchestrated; two are not.

12.5 Failure handling

Every stage carries retries with exponential backoff and a Catch handler routing to a single failure path: NotifyFailure publishes to SNS, then PipelineFailed terminates the execution. The two asynchronous stages poll their own status — starting a load is not the same as finishing one, and treating the API call's success as the stage's success is a classic orchestration bug.

Part 13 — EventBridge

13.1 Concepts

- **Event** — a JSON record describing something that happened.
- **Event bus** — where events are published. The default bus receives AWS service events free.
- **Rule** — a pattern that matches events and routes them to targets.
- **Target** — what the rule invokes — here, an SNS topic.
- **Schedule** — rules can also fire on a cron or rate expression.

13.2 How it is used here — and the distinction that matters

One rule, matching Glue Job State Change events with FAILED, TIMEOUT or ERROR, targeting the SNS alerts topic.

Its value is coverage the state machine cannot provide: it catches Glue jobs that fail when started OUTSIDE an orchestrated run — a manual run, or the two jobs that are not Step Functions stages at all.

```
PATH A — orchestrated failure
  Step Functions state fails -> Catch -> NotifyFailure -> SNS topic -> email

PATH B — any Glue job failure, orchestrated or not
  Glue emits 'Glue Job State Change' -> EventBridge rule -> SNS topic -> email
```

```
PATH C — metric threshold breach
  CloudWatch alarm changes state -> alarm action -> SNS topic -> email

Three INDEPENDENT paths to the same topic.
CloudWatch alarms do NOT flow through EventBridge.
```

Figure 9 — three separate routes to the same alerting topic.

Say this precisely: 'alarms invoke the SNS topic directly as an alarm action. EventBridge is a separate path for job-level events. They both end at SNS, but neither passes through the other.'

Part 14 — CloudWatch and SNS

14.1 The three CloudWatch products

- **Logs** — streams of text from services and applications, grouped into log groups.
- **Metrics** — numeric time series, identified by namespace, name and dimensions.
- **Alarms** — evaluate a metric over a period and transition between OK, ALARM and INSUFFICIENT_DATA, invoking actions on transition.

14.2 What is monitored here

VERIFIED — Four alarms exist, all currently OK, all with one alarm action pointing at the SNS topic.

Alarm	Watches for
pipeline-execution-failed	A failed Step Functions execution
pipeline-execution-timed-out	An execution exceeding its timeout
pipeline-execution-duration-high	An execution running unusually long
redshift-compute-seconds-high	Warehouse capacity consumption above threshold

A CloudWatch dashboard covers executions, per-job Glue metrics and Redshift capacity. Step Functions logs at full detail to CloudWatch Logs.

14.3 Why there is no alarm on Glue task failures

The Glue task-failure metric carries a per-run dimension, so covering all runs would require a metric search expression — and CloudWatch alarms reject search expressions. Job failure is alarmed through the EventBridge rule and the state machine's Catch instead.

This is the better signal anyway: a failed Spark task can be retried by Spark and the job still succeeds. Alarming on task failures would page you for events that resolved themselves.

14.4 pipeline_runs

A table in RDS mirroring Glue job run history, populated by the sync-pipeline-runs Python Shell job. It gives run history a queryable, durable home alongside the master data rather than only in Glue's own API.

14.5 The SNS confirmation caveat

TRAP — Terraform can create an email subscription; it cannot confirm one. The subscription is created in PendingConfirmation state and AWS emails a link. Until someone clicks it, NOTHING is delivered — and a successful deployment proves only that the subscription exists. The confirmed state is what to verify, and the real proof is publishing a test message and receiving it.

VERIFIED — The subscription on this platform is confirmed and was proven by a test publish. All four alarms are wired to that topic.

Part 15 — Secrets Manager and KMS

15.1 Secrets Manager

Stores a credential and returns it to authorised callers at runtime. The critical design idea is the split between the identifier and the value:

The identifier is not sensitive; the value is. A secret ARN in a configuration file, a log line or a screenshot is harmless. A password in the same place is an incident. Designing so that only ARNs travel is what makes the rest of the system safe to debug in public.

VERIFIED — Two secrets exist: the RDS master credential and the Redshift admin credential. The RDS master password is GENERATED by Terraform at deploy time, so its value exists only in state and in the secret — never in a variables file, source file or output.

NOT IMPLEMENTED / DELIBERATELY NOT BUILT — Automatic rotation is not configured. Rotation requires a rotation function with network access to the database and coordinated updates to every consumer. For a platform with two consumers and a single operator, the added machinery was judged not worth it — but it is the obvious next hardening step.

15.2 KMS in this project

Resource	Encryption	Key type
S3 data lake default encryption	SSE-KMS	Customer-managed CMK
Athena query results	SSE-KMS	Customer-managed CMK
Both Secrets Manager secrets	SSE-KMS	Customer-managed CMK
Step Functions log data	SSE-KMS	Customer-managed CMK
Terraform state bucket	SSE-S3 (AES256)	S3-managed
RDS storage	Encrypted	AWS-managed key
Redshift namespace	Encrypted	AWS-managed key

The CMK has automatic rotation enabled.

The limitation, stated honestly: RDS storage and the Redshift namespace use AWS-managed keys rather than the project CMK. WHAT: they are encrypted, but with keys whose policy cannot be edited. WHY IT EXISTS: both were created before the CMK. WHY IT WAS CHOSEN: switching the encryption key on either requires REPLACING the resource — a snapshot-restore for RDS and a rebuild for the namespace. WHAT WOULD CHANGE IT: accepting that replacement during a maintenance window.

TRAP — A CMK has BOTH a key policy and IAM permissions, and both must allow the operation. A role with kms:Decrypt in its IAM policy still fails if the key policy does not permit it. This platform's key policy delegates to the account root, so IAM grants are sufficient.

Part 16 — VPC and networking

16.1 The actual topology — verified

VERIFIED — Live inspection of subnets, route tables and the RDS subnet group. The findings below correct a common and comfortable assumption.

```
VPC (default VPC of the account)
|
+-- RDS subnet group: 2 subnets, AZ-a and AZ-b
|     route table: MAIN, which has an INTERNET GATEWAY route
|     map_public_ip_on_launch = true
|     => these are PUBLIC SUBNETS
|     RDS instance: publicly_accessible = FALSE
|        -> no public IP, no publicly resolvable endpoint
|        -> security group admits ONLY the Glue security group on 5432
|
+-- Glue private subnet: 1 subnet, AZ-a  (created by the network module)
      route table: dedicated, routes 0.0.0.0/0 to the NAT GATEWAY
      => this is a PRIVATE SUBNET
      NAT gateway  -> outbound internet for runtime pip installs
      S3 gateway endpoint -> lake traffic stays inside AWS, off the metered NAT
```

Figure 10 — the real network layout, including the part that is easy to get wrong.

16.2 publicly_accessible = false is NOT the same as being in a private subnet

This is the single most important networking nuance in this platform, and it is worth being precise about because an interviewer may probe it.

Claim	True?	Why
The RDS instance has no public address	TRUE	publicly_accessible = false means AWS assigns no public IP and the endpoint does not resolve publicly
The RDS instance is in a private subnet	FALSE	Its subnets route to an internet gateway and auto-assign public IPs — they are public subnets
The RDS instance is unreachable from the internet	TRUE	No public address to reach, and the security group admits only the Glue security group
The RDS instance is protected by network isolation	FALSE	It is protected by addressing and by the security group, not by subnet routing

How to say it well: 'RDS is not publicly accessible, but I want to be precise — it sits in subnets that route to an internet gateway. Its protection comes from having no public address and from a security group that admits only the Glue security group, not from subnet isolation. Moving it to genuinely private subnets would be a defence-in-depth improvement.'

WHAT WOULD BE REQUIRED TO CHANGE IT: create private subnets in at least two AZs, create a new DB subnet group, and modify the instance to use it — which triggers a brief outage as the instance's network placement changes.

16.3 Why Glue needs network configuration at all

By default a Glue job runs in AWS-managed networking with internet access and no route into your VPC. That is fine for jobs that only touch S3. A job that must reach a private database has to be placed inside the VPC, which is what the NETWORK Glue connection does.

The moment it joins a private subnet, it loses internet access — hence the NAT gateway, because both database-facing Spark jobs install a PostgreSQL driver at runtime.

And because bulk lake traffic would otherwise be billed through the NAT, an S3 gateway endpoint routes it inside AWS instead.

Path	Route	Cost
Glue -> RDS	Inside the VPC, security-group to security-group	None
Glue -> PyPI	Private subnet -> NAT gateway -> internet	NAT hourly + per GB
Glue -> S3	Private subnet -> S3 gateway endpoint	Free

The cost lesson worth carrying anywhere: the S3 gateway endpoint is free and the NAT gateway is metered per GB. Routing bulk S3 traffic through the endpoint rather than the NAT is the single most common avoidable cost in private-subnet data architectures.

Interview questions on this

Q Is your RDS instance private?

It has no public address — publicly_accessible is false — and its security group admits only the Glue security group. But to be precise, its subnets route to an internet gateway, so they are public subnets. The protection is addressing and security groups, not subnet isolation. Genuinely private subnets would be a defence-in-depth improvement.

Q Why do you need a NAT gateway?

Both database-facing Glue jobs run inside a private subnet to reach RDS, and they install a PostgreSQL driver at runtime. A private subnet has no internet route, so the NAT provides outbound-only access without making the subnet reachable from outside.

Q Why an S3 gateway endpoint?

Without it, all lake traffic from those jobs would traverse the NAT and be billed per GB. The gateway endpoint keeps it inside AWS at no charge. It is also a security improvement — that traffic never touches a public path.

Part 17 — IAM and the OIDC story

17.1 The roles in this platform

Role	Assumed by	May do
Glue execution role	Glue jobs	Read one secret ARN, read/write lake prefixes, write its log group, manage ENIs for VPC jobs
Step Functions role	The state machine	Start the four orchestrated Glue jobs, call the Redshift Data API, refresh one QuickSight dataset, publish to SNS, read one secret
Redshift role	Redshift	Read the warehouse prefix in S3 for COPY
EventBridge role	The event rule	Publish to the SNS topic
CI plan role	GitHub Actions on pull requests	ReadOnlyAccess, plus read state and two explicitly named resources
CI apply role	GitHub Actions from the production environment	PowerUserAccess, plus IAM management scoped to this project's name prefixes, plus state write

17.2 GitHub OIDC end to end

```
1. Workflow job starts, declares  permissions: id-token: write
2. GitHub mints a signed JWT describing the run:
       sub = repo:<owner>/<repo>:pull_request            (plan)
       sub = repo:<owner>/<repo>:environment:production  (apply)
       aud = sts.amazonaws.com
3. Action calls sts:AssumeRoleWithWebIdentity with that token
4. STS validates the signature against the registered OIDC provider
5. STS evaluates the role's TRUST POLICY conditions on sub and aud
6. Temporary credentials returned, valid for the job only

Environment variables override repository variables, which is what routes
each workflow to the correct role.
```

Figure 11 — no stored AWS credential at any point.

17.3 The immutable-ID subject problem — a real incident

Authentication failed with 'Not authorized to perform sts:AssumeRoleWithWebIdentity', while the trust policy matched the documented subject format exactly.

CloudTrail recorded the identity actually presented:

```
repo:<owner>@<numeric-account-id>/<repo>@<numeric-repo-id>:pull_request
```

GitHub had issued the subject in an IMMUTABLE-ID form carrying numeric account and repository IDs, so that renaming an account or repository cannot be used to impersonate it. A StringEquals condition on the classic owner/name form could never match it.

The fix lists BOTH exact strings in the condition:

```
values = [
  "repo:${var.github_repository}:pull_request",
  "repo:${var.github_repository_immutable}:pull_request",
]
```

StringEquals matches ANY element of a list, so this remains an exact match against one repository — it is not a wildcard and does not widen the trust boundary. Both forms are accepted because which one GitHub sends is GitHub's decision and can change.

> **The transferable lesson:** when an identity check fails, read what was actually presented rather than reasoning about what should have been. The caller's error describes the outcome; the audit record describes the cause. This is the best troubleshooting story in the project.

Interview questions on this

Q How does GitHub authenticate to AWS without storing credentials?

OIDC federation. GitHub mints a short-lived signed token describing the repository and workflow context. AWS validates it against a registered identity provider and checks the role's trust policy conditions on the subject and audience claims, then issues temporary credentials. Nothing long-lived is ever stored.

Q Why separate plan and apply roles?

The blast radius differs by an order of magnitude. A pull request — including from a fork — can only ever reach the read-only plan role. The apply role's trust policy requires the protected production environment in the subject claim, so approval is enforced by AWS rather than by workflow convention.

Q What was the hardest problem you hit?

An OIDC failure where the trust policy looked correct. CloudTrail showed GitHub was issuing the subject with immutable numeric IDs, which the documented format did not mention. The error message could not have revealed it — only the audit record could.

Part 18 — Terraform

18.1 The vocabulary

Construct	What it does
provider	Configures a target API — here, AWS in one region
resource	A thing to create and manage
data source	Reads something that already exists, without managing it
variable	An input, optionally with a default and validation
output	A value exported from a module or the root
locals	Named expressions computed once and reused
module	A reusable grouping of resources with its own inputs and outputs
count / for_each	Create N copies, or one per element of a map or set
conditional resource	count = var.flag ? 1 : 0 — the standard on/off switch
depends_on	An explicit ordering edge when there is no implicit reference
import	Bring an existing resource under management without recreating it

18.2 This project's structure

VERIFIED — Eleven modules are wired into the root configuration: s3, iam, cloudwatch, rds, redshift, network, kms, secrets, stepfunctions, monitoring, github_oidc.

The root configuration directly declares what does not justify a module: the Glue jobs and crawlers, the catalog databases, the Athena workgroup, the Glue IAM role, the state bucket and the CI wiring.

```
terraform/
  main.tf  security.tf  glue_jobs.tf  glue_crawlers.tf  athena.tf  backend.tf
  variables.tf  outputs.tf  versions.tf
  modules/
     s3  iam  cloudwatch  rds  redshift  network  kms  secrets
     stepfunctions  monitoring  github_oidc

Dependency direction (simplified):
     kms  -->  secrets  -->  stepfunctions
     kms  -->  s3
     network  -->  glue jobs (via the NETWORK connection)
     secrets + network  -->  github_oidc  (plan role needs to READ them during refresh)
```

Figure 12 — module layout and the dependency edges that matter.

18.3 State

Terraform state maps configuration to real resources. It records what exists, its attributes, and the dependency graph. Without it, Terraform cannot tell 'create this' from 'this already exists'.

VERIFIED — Remote state in a private, versioned S3 bucket. Locking uses S3 CONDITIONAL WRITES (use_lockfile). DynamoDB locking is NOT used and no lock table exists.

Why state must never be committed: it contains every attribute of every resource — including generated passwords and secret values in plaintext. It is gitignored, and CI enforces this with a forbidden-file guard that fails the build if a state file is ever tracked.

18.4 The plan/apply model

```
terraform plan     reads state, refreshes real resources, computes a diff
                   changes NOTHING
terraform apply    executes the diff
terraform destroy  removes everything under management
terraform import   adopts an existing resource into state

Plan output is the change-control artifact: 0 to add, 2 to change, 0 to destroy
is a reviewable statement about what will happen.
```

Figure 13 — the plan is the review surface.

TRAP — A plan that proposes recreating something you did not touch is a signal, not noise. In this project it happened twice: once from an encryption change altering how S3 reports object integrity, and once from line endings changing a file hash. Both were investigated rather than applied. See Part 23.

Part 19 — GitHub Actions and CI/CD

19.1 The three workflows

Workflow	Trigger	AWS access	What it does
ci.yml	PR and push to main	NONE	terraform fmt -check, init -backend=false, validate, compileall, pytest, gitleaks, forbidden-file guard
terraform-plan.yml	PR to main (path-filtered)	OIDC, read-only	Full plan against real infrastructure, posted to the job summary
terraform-apply.yml	workflow_dispatch only	OIDC, write	Plan to a file, then apply that exact file

19.2 Why CI needs no AWS credentials

Deliberate. The checks that gate a pull request are all static: formatting, configuration validity, Python syntax, unit tests and secret scanning. None needs cloud access.

Two benefits fall out. A pull request cannot be blocked by an AWS outage or a permissions problem. And a pull request from a fork — which GitHub never grants an OIDC token — still gets full static validation.

terraform-plan.yml is separate precisely because it is the one pull-request workflow that does need AWS, and isolating it keeps the credential-bearing surface small.

19.3 Why apply has stronger permissions and stricter gates

Plan only reads. Apply creates, modifies and deletes — including IAM. So apply is gated three ways: a human dispatches it, a typed confirmation must read exactly APPLY, and the protected production environment must approve. The environment name is part of the OIDC subject claim, so that approval is enforced by the AWS trust policy rather than by workflow convention.

A limitation worth volunteering: the environment approval gates the JOB, not the DIFF. A reviewer approves before the plan has been computed. Splitting plan and apply into two jobs with the environment on the second would let a reviewer see the actual diff first. Logged as an improvement.

19.4 TFVARS delivery

terraform.tfvars is gitignored, so the runner has no variable values at all. Both AWS workflows materialise it from a repository secret, then delete it in an always-run cleanup step.

> **Why the WHOLE file rather than a variable per secret:** a plan is only worth reading if its inputs match the operator's exactly. Supplying a subset lets Terraform fall back to defaults for the rest — producing a diff that looks authoritative while proposing changes that are not real. That is worse than no plan.

> **TRAP —** An operational coupling this creates: if a value in terraform.tfvars changes, the GitHub secret must be updated too. Otherwise the next plan proposes undoing the change — and an apply would carry it out.

19.5 The bootstrap problem

When the OIDC trust policy itself is broken, no workflow can fix it. The plan workflow is read-only by design and cannot grant its own permissions, and the apply workflow authenticates through the same broken trust policy.

So the first apply of an OIDC trust policy is necessarily local. Every apply after that can go through the pipeline.

> **Good answer to 'how do you bootstrap CI/CD?':** 'the credential path has to exist before automation can use it, so the first apply is local by necessity. After that everything flows through the pipeline — and I verified the apply path end to end with a zero-change apply, which exercises the whole chain without being able to alter anything.'

19.6 Secret hygiene

- **gitleaks** — scans full history, not just the tip, on every run.
- **Forbidden-file guard** — a dependency-free check failing the build if a state file, tfvars, plan file, key, .env or bytecode is ever tracked.
- **No plan file uploaded** — a saved plan embeds sensitive values in plaintext, so it is never an artifact.
- **Generated database password** — never in tfvars, source or an output.

Part 20 — Data quality and reconciliation

20.1 Why 'the job succeeded' proves nothing

A job exits zero when its code ran without throwing. It says nothing about whether the output is correct. Reconciliation is computing the same number two independent ways and requiring agreement.

20.2 The validated figures

Metric	Value
Silver rows / fact_trips / SPICE rows	2,851,125
dim_zone / zone_matches / taxi_zones	265
dim_date (gap-free calendar)	33
dim_payment (full TLC code set)	7
SUM(fare_amount)	52,389,612.60
SUM(total_amount)	79,198,239.40
Gold: daily / vendor / borough / payment / hourly	33 / 95 / 203,611 / 128 / 748
Distinct vendors	3

18 of 18 in-warehouse checks pass. 33 of 33 days tie out against Gold computed independently in Athena, on both trip count and revenue. SPICE ingests 2,851,125 rows with 0 dropped.

The design choice that makes this meaningful: fact_trips is a LOSSLESS projection of Silver — no filter, no deduplication, no enrichment — specifically so its row count MUST equal Silver exactly. If it filtered anything, an equal count would prove nothing.

20.3 Deliberately unfiltered anomalies

TRAP — 321 Silver rows have a negative total_amount, and one trip records $863,380.37. These are carried through deliberately. WHAT: implausible source values. WHY IT EXISTS: Silver validates fare_amount but not total_amount. WHY IT WAS CHOSEN: filtering them would break the Silver-to-fact row-count reconciliation, which is the platform's strongest correctness claim. WHAT WOULD CHANGE IT: a quarantine table capturing rejected rows, so counts still reconcile as kept + rejected = source.

That trade — keep the anomalies and keep the proof, or clean the data and weaken the proof — is a good thing to be able to discuss.

Part 21 — QuickSight

21.1 Concepts

Concept	Meaning
Data source	The connection to a database
Dataset	The modelled data — joins, calculated fields, types
SPICE	In-memory cache. The dashboard reads from it, not from the warehouse
Direct query	The alternative — every visual hits the warehouse live
Analysis	The editable authoring surface
Dashboard	The published read-only result
VPC connection	How QuickSight reaches a database with no public address

21.2 What exists here

VERIFIED — Six resources: an IAM role for the VPC connection, the VPC connection, a Redshift data source, a SPICE dataset (5 physical tables, 9 logical tables, 37 output columns, ~1.9 GB SPICE), an analysis and a dashboard of 5 sheets and 28 visuals.

The dashboard reads Redshift over the VPC connection. SPICE ingestion is the final stage of the orchestrated pipeline, so the cache refreshes as part of every run.

21.3 Managed outside Terraform

MANUAL / OUTSIDE TERRAFORM — All six QuickSight resources are created through the API and are NOT in Terraform state. Verified: zero QuickSight resources appear in terraform state list.

WHAT IT IS: the one plane where a rebuild from code alone would not reproduce the system.

WHY IT EXISTS: the dashboard and analysis definitions are roughly 31 KB of deeply nested JSON each — 5 sheets, 28 visuals, 6 calculated fields, 5 filter groups. In Terraform that is thousands of lines per resource.

WHY IT WAS CHOSEN: three reasons.

17. **The credential cannot be imported** — the data source stores a username/password pair inside QuickSight rather than referencing a secret, and QuickSight never returns that password. Terraform cannot import it faithfully.
18. **There is no round trip** — move one chart in the UI and the code is stale. Recovering means re-exporting JSON and hand-translating it.
19. **The risk is asymmetric** — an imperfect dataset import rewrites the dataset and forces a full re-ingest of 1.9 GB.

WHAT WOULD BE REQUIRED TO CHANGE IT: the defensible split is Terraform for the infrastructure — IAM role, VPC connection, data source, dataset, adopted via import blocks — and QuickSight asset bundle exports, committed to the repository, for the analysis and dashboard. Asset bundles are AWS's own mechanism for versioning and promoting BI content.

> **How to present this:** 'connection and dataset are infrastructure; dashboards are content authored in a GUI. I documented the split rather than generating thousands of lines of HCL that would drift the first time someone moved a chart.' A reasoned boundary reads far better than an unexplained gap — but only if you volunteer it.

Part 22 — One complete end-to-end run

The most important chapter. Follow a new trip file from arrival to dashboard.

Step 1 — The file lands

New trip Parquet is placed under bronze/transactions/. Bronze is never edited; a new file is simply new input.

> **NOT IMPLEMENTED / DELIBERATELY NOT BUILT —** There is NO automatic trigger on object arrival. No S3 event notification, no EventBridge schedule starting the pipeline. Execution is started deliberately. For a batch pipeline over periodically published files, on-demand execution is the honest design — and it avoids a partially uploaded file starting a run.

Step 2 — Execution starts

A Step Functions execution begins at SilverETL.

Step 3 — SilverETL

Invoked by	Step Functions, Glue startJobRun.sync — waits for completion
Reads	bronze/transactions/
Does	Cleans, validates, types; validates fare_amount
Writes	silver/ as Delta, partitioned by pickup year and month
Network	Not in the VPC — touches only S3
On failure	Retries with backoff; then Catch to NotifyFailure

Step 4 — GoldETL

Reads	silver/
Does	Computes five aggregations
Writes	gold/ as five Delta tables

Network	Not in the VPC

Step 5 — GoldenZoneETL

Reads	The zone CSV under bronze/reference/
Does	Standardises each row, computes record_hash, calls sp_upsert_golden_zone
Writes	RDS: INSERTED / NO_CHANGE / UPDATED per zone
Network	IN the VPC — private subnet, security-group-to-security-group to RDS
Credential	Resolves the secret ARN at runtime; connects over TLS

Step 6 — WarehouseExport

The convergence point, and the only job reading both pipelines.

Reads	silver/ AND the mastered zones in RDS
Resolves zones	By BUSINESS KEY through taxi_zones, never by version pointer
Writes	warehouse/ — fact_trips, dim_zone, dim_date, dim_payment, plain Parquet, unpartitioned
Network	IN the VPC
Validates	dim_zone must contain exactly 265 rows — the job fails if not

Step 7 — StartRedshiftCopy and its poll loop

Step Functions calls the Redshift Data API with the 13 statements read from the SQL file at deploy time: 4 TRUNCATE, 4 COPY, 4 ANALYZE, 1 SELECT. Then WaitForRedshiftCopy and CheckRedshiftCopy loop until the batch finishes, routing to RedshiftCopyComplete or RedshiftCopyFailed.

> **TRAP —** This is where the non-atomic load matters. If a COPY fails after a TRUNCATE has committed, the warehouse is left partially refreshed. Recovery is re-running the same file.

Step 8 — StartSpiceRefresh and its poll loop

Step Functions calls QuickSight createIngestion, then polls until the ingestion completes. SPICE ingests 2,851,125 rows with 0 dropped.

Step 9 — PipelineSucceeded

The execution terminates successfully. Its full history remains inspectable.

Step 10 — What was logged along the way

- Every state transition in the Step Functions execution history, at full detail, to CloudWatch Logs
- Each Glue job's driver and executor logs to its CloudWatch log group
- Redshift Data API statement status, polled and recorded in the execution
- pipeline_runs in RDS, updated separately by sync-pipeline-runs — NOT part of this execution

Step 11 — What happens when something fails

```
Any state fails
   |
```

```
    |  retries with exponential backoff first
    |  still failing?
    v
Catch  -->  NotifyFailure   publishes to the SNS topic
                |
                v
           PipelineFailed   execution terminates as FAILED

In PARALLEL and INDEPENDENTLY:
   Glue emits a Job State Change event  ->  EventBridge rule  ->  same SNS topic
   CloudWatch alarm on failed executions ->  alarm action     ->  same SNS topic

SNS  ->  confirmed email subscription  ->  a person finds out
```

Figure 14 — three independent detection paths converging on one topic.

Multiple paths are not redundancy for its own sake. The Catch handles orchestrated failures. EventBridge covers jobs run outside the pipeline. The alarm covers cases where the execution itself never reaches its Catch — a timeout, for instance.

Part 23 — Troubleshooting scenarios

Each of these is either an incident that actually occurred in this project or a realistic failure of a component that exists. Where it actually happened, it says so.

23.1 OIDC AssumeRoleWithWebIdentity fails [actually occurred]

Symptom	Workflow fails at Configure AWS credentials: 'Not authorized to perform sts:AssumeRoleWithWebIdentity'
Likely cause	The presented sub claim does not match the trust policy — often the immutable-ID form, or the wrong role ARN for the context
Where to look	CloudTrail: lookup AssumeRoleWithWebIdentity, read userIdentity.principalId for the SUBJECT ACTUALLY PRESENTED. The workflow log will not show it
Safe fix	Add the presented subject as an additional exact StringEquals value; verify environment vs repository variable routing
Do NOT	Do not relax the condition to a wildcard such as repo:*. That would let any repository assume the role

23.2 Terraform plan proposes recreating S3 script objects [actually occurred]

Symptom	Every plan shows the same N objects changing, on files nobody edited
Likely cause	Line endings: CRLF on a Windows checkout, LF on a Linux runner, so filemd5 differs for identical source. (A related variant: SSE-KMS changes what S3 returns as the object ETag)
Where to look	Compare the hash of the on-disk file against the state value; check whether the committed blob differs from the working tree only by line endings
Safe fix	Add .gitattributes with eol=lf, normalise the working tree, apply once to converge. For the ETag variant, compare against source_hash instead of etag
Do NOT	Do not apply repeatedly hoping it settles — it will alternate forever between platforms

23.3 Glue job cannot reach RDS

Symptom	Job times out connecting to the database endpoint
Likely cause	Job not attached to the NETWORK connection, or the RDS security group does not admit the Glue security group, or the job is in the wrong subnet
Where to look	Glue job definition: is the connection attached? RDS security group inbound rules on 5432. VPC Flow Logs if enabled
Safe fix	Attach the connection; ensure the RDS security group admits the Glue security group as a source rather than a CIDR
Do NOT	Do not open the RDS security group to 0.0.0.0/0 to test connectivity

23.4 Secrets Manager AccessDenied from a Glue job

Symptom	GetSecretValue denied at runtime
Likely cause	The Glue role lacks GetSecretValue on that exact ARN, or lacks kms:Decrypt on the CMK encrypting it
Where to look	The job's CloudWatch log group for the exact denial message — it names the action and resource
Safe fix	Grant GetSecretValue on the specific secret ARN, and kms:Decrypt on the CMK
Do NOT	Do not grant secretsmanager:* on all resources

23.5 Terraform plan fails: glue:GetConnection or GetSecretValue denied [actually occurred]

Symptom	CI plan fails during refresh with AccessDenied on those two actions
Likely cause	The ReadOnlyAccess managed policy deliberately excludes both, because each can return credential material
Where to look	The plan job log; confirm with CloudTrail which action and resource were denied
Safe fix	Add a narrowly scoped statement for the exact secret ARNs and the exact connection ARN. Glue also requires the catalog ARN alongside the connection ARN
Do NOT	Do not attach a broader managed policy to make it go away

23.6 Redshift COPY fails part-way [actually occurred]

Symptom	Load fails; some warehouse tables populated, others empty
Likely cause	TRUNCATE commits implicitly, so earlier statements are already durable when a later one fails
Where to look	Redshift STL_LOAD_ERRORS for the row-level cause; the Step Functions execution history for which statement failed
Safe fix	Fix the cause, then re-run the same load file — it is idempotent by design
Do NOT	Do not assume the warehouse rolled back. It did not

23.7 Redshift ANALYZE permission denied [actually occurred]

Symptom	Load fails on ANALYZE even though the user has full table privileges
Likely cause	ANALYZE is reserved to the table owner and cannot be granted
Where to look	The Data API statement error for the failing statement
Safe fix	Run the load as the owning identity — the orchestrator authenticates as the admin via a Secrets Manager ARN
Do NOT	Do not transfer table ownership casually to work around it

23.8 dim_zone row count wrong (264 instead of 265) [actually occurred]

Symptom	Warehouse export job fails its own validation
Likely cause	Current-record resolution joined on the SCD2 version pointer, which is never repointed, dropping every zone that has ever been updated
Where to look	The export job's zone query; compare a business-key resolution count against the pointer-based count
Safe fix	Resolve by business key: zone_matches to taxi_zones to golden_zones WHERE is_current
Do NOT	Do not relax the expected count to 264. The count was the symptom, not the problem

23.9 SNS alerts never arrive [actually occurred]

Symptom	Alarms transition to ALARM but no email arrives
Likely cause	The email subscription is in PendingConfirmation — Terraform can create it but cannot confirm it
Where to look	SNS subscription state: a real ARN means confirmed; the literal string PendingConfirmation means not
Safe fix	Click the confirmation link, then prove it with aws sns publish
Do NOT	Do not treat successful deployment as evidence that alerting works

23.10 Step Functions execution fails

Symptom	Execution ends FAILED
Likely cause	Any stage's underlying job failed after exhausting retries
Where to look	Execution history — it names the failing state and its error. Then that job's CloudWatch log group
Safe fix	Fix the underlying job and start a new execution
Do NOT	Do not edit the state machine to skip the failing stage

23.11 Terraform state lock error

Symptom	Error acquiring the state lock
Likely cause	A concurrent operation, or a previous run killed before releasing it
Where to look	The state bucket's lock object; confirm no CI apply is in flight
Safe fix	Wait for the other operation. Only force-unlock after confirming nothing is running
Do NOT	Do not force-unlock reflexively — you can corrupt state mid-write

23.12 QuickSight cannot connect to Redshift

Symptom	Data source connection test fails
Likely cause	VPC connection unavailable, Redshift security group does not admit it, or the stored credential is stale
Where to look	QuickSight VPC connection status; Redshift security group inbound rules
Safe fix	Confirm the VPC connection is AVAILABLE and the security group admits it
Do NOT	Do not make the Redshift workgroup publicly accessible to work around it

23.13 Athena query fails or returns nothing

Symptom	Table not found, or zero rows from data you know exists
Likely cause	The crawler has not run since new partitions appeared, or the Silver crawler was configured with an S3 target rather than a Delta target
Where to look	Glue Data Catalog table definition and its partition list; crawler run history
Safe fix	Re-run the crawler; ensure the Delta table uses a delta_target
Do NOT	Do not hand-edit catalog partitions as a routine fix

Part 24 — Design decisions

Decision	Alternatives	Why chosen	Trade-off
S3 as the data lake	RDS, Redshift-only	Cheap at volume, schema-on-read, engine-independent	No transactions without a table format
Bronze / Silver / Gold	One transformed dataset	Each layer reproducible; failures trace to a layer	More storage, more jobs
Delta on Silver and Gold	Plain Parquet	ACID writes, time travel, schema evolution	Only Delta-aware readers
Plain Parquet for warehouse/	Delta	Redshift COPY cannot read Delta	A second copy of the data
RDS PostgreSQL for MDM	Redshift, DynamoDB, S3	Atomic compare-and-version for SCD2	A continuously billed instance
Redshift for analytics	Athena-only	Predictable interactive latency for BI	Fixed namespace storage cost
Glue	EMR, Lambda, ECS	Serverless Spark, no cluster to operate	Less control over runtime
Step Functions	Airflow, Lambda chaining, cron	Declarative retries, durable history, no server	AWS-specific definition
QuickSight	Grafana, Superset, QuickSight-free	Native VPC and Redshift integration, SPICE	Per-user subscription cost
Terraform	CloudFormation, CDK, console	Reviewable plan diffs, module reuse	State must be managed
S3 remote state	Local state	Shared, versioned, recoverable	Bootstrap ordering to solve
S3 conditional-write locking	DynamoDB lock table	No extra resource to run or pay for	Newer mechanism
OIDC federation	Long-lived access keys	No credential to leak or rotate	Trust-policy subtlety, as encountered
Manual apply	Auto-apply on merge	Protects a reconciled warehouse and live dashboard	Deployment needs a human
DISTSTYLE EVEN on fact	DISTKEY on a zone	Dimensions already replicated; two zone joins; real skew	No co-location benefit
No surrogate trip key	IDENTITY column	Non-deterministic across slices would renumber every reload	No single-column trip identifier
Business-key resolution	Version-pointer join	Always finds the current version; survives updates	One extra join
Vendor MDM not active	Master vendors too	No authoritative source; would invent master data	No vendor dimension
QuickSight outside Terraform	Full HCL definitions	31 KB JSON per definition; no round trip; re-ingest risk	Not reproducible from code alone
Full reprocessing, no bookmarks	Incremental with bookmarks	Every layer reproducible; row-count reconciliation stays simple	Longer runtime

Part 25 — What is deliberately not built

Stating these confidently is a strength. Each is a decision, not an omission.

Not built	Why
API Gateway / Lambda serving layer	An earlier design sketched it for master-data lookups. Never deployed; the scaffolding has been removed. Consumers reach mastered data through the warehouse
Active vendor mastering	No authoritative vendor source exists and the data contains an unnameable vendor code. The old vendor artifacts are frozen and referenced by nothing
dim_vendor	Would fabricate master data inside an MDM project. vendorid is a degenerate dimension on the fact table
Fuzzy matching / survivorship	One authoritative source with one row per location_id. There is no ambiguity to resolve
Surrogate trip key	No stable natural key exists, nothing downstream needs one, and an IDENTITY column would renumber every trip on reload
QuickSight in Terraform	31 KB of nested JSON per definition, an unimportable credential, no round trip, and re-ingest risk
Glue bookmarks / incremental	Full reprocessing keeps every layer reproducible and keeps row-count reconciliation meaningful
Automatic pipeline trigger	Batch pipeline over periodically published files. On-demand execution avoids a partial upload starting a run
Multi-AZ RDS	Master data is rebuildable and backups cover history. Would roughly double instance cost
Secret rotation	Two consumers and one operator. The obvious next hardening step, not a current gap
Kafka, Airflow, EMR, ECS, EKS	Batch pipeline running on demand. Step Functions and Glue cover it without a platform to operate
Streaming ingestion	The source is published as periodic files

Part 26 — Interview preparation

26.1 Architecture and design

Interview questions on this

Q Walk me through your architecture.

Two pipelines with opposite characteristics that converge. 2.85M trips flow Bronze to Silver to Gold in S3 with Delta. 265 taxi zones flow into RDS PostgreSQL through an SCD Type 2 stored procedure. A warehouse export job reads both and writes a plain-Parquet star schema, which Redshift COPYs and QuickSight ingests into SPICE. One Step Functions execution orchestrates four Glue jobs plus the load and the refresh.

Q Why two storage systems instead of one?

They solve opposite problems. S3 is cheap, reprocessable and engine-independent — right for millions of disposable trip rows. RDS gives atomic transactions — required because the SCD2 compare-and-version step must not race. Using one for both would sacrifice either cost or correctness.

Q What would you improve with another month?

Three things in order. Redshift load atomicity via staging-and-swap, because that is a real correctness gap. QuickSight infrastructure into Terraform with asset bundles for the dashboards. Then moving RDS into genuinely private subnets for defence in depth.

26.2 Data engineering

Interview questions on this

Q How do you know your pipeline is correct?

Reconciliation, not exit codes. 18 in-warehouse checks plus 33 days of counts and revenue tied out against Athena computing the same totals from Gold independently. fact_trips is a lossless projection of Silver specifically so its row count must match exactly.

Q How do you handle bad data?

In this build, deliberately not by filtering. 321 rows have a negative total and one trip is $863,380. Filtering would break the row-count reconciliation, which is the strongest correctness claim I have. The right long-term fix is a quarantine table so counts still reconcile as kept plus rejected.

Q Why medallion rather than one transformation?

Diagnosis. When a number is wrong you trace backwards through named, individually reproducible layers. It also means a bad cleaning rule is fixable — Bronze is untouched, so you fix the rule and reprocess.

26.3 Rapid-fire answers

Question	Answer
Why S3 not RDS for the lake?	Cost at volume, schema-on-read, engine independence
Why both RDS and Redshift?	OLTP for atomic SCD2 versioning; OLAP for scan-and-aggregate serving
Why not everything in Redshift?	Expensive per GB, reprocessing becomes reloading, only Redshift could read it
Why Step Functions?	Declarative retries, error routing, durable inspectable history, no server
Why Glue?	Serverless Spark for an on-demand batch pipeline; no cluster lifecycle
Why crawlers?	Keep partitions current automatically; hand-written definitions drift silently
Why Athena if QuickSight uses Redshift?	Independent engine for reconciliation, plus ad-hoc analysis
Is RDS private?	No public address and a tight security group — but its subnets route to an IGW
Why NAT Gateway?	VPC-attached Glue jobs install a driver at runtime and have no internet route
Why S3 gateway endpoint?	Keeps bulk lake traffic off the metered NAT, at no charge
Why Secrets Manager?	Jobs get an ARN, never a password; the value never enters config or source
Why OIDC not access keys?	No long-lived credential exists to leak or rotate
Why separate plan and apply roles?	Blast radius differs; a PR can only ever reach read-only
Why remote state?	Shared, versioned, recoverable; local state cannot be reviewed or shared
DynamoDB for locking?	No — S3 conditional writes. No lock table exists
Why DISTSTYLE EVEN?	Dimensions are already replicated; two zone joins; hashing on zone would skew
How do you prevent secrets reaching Git?	gitignore, gitleaks over full history, a forbidden-file guard, and generated passwords
Hardest problem?	OIDC immutable-ID subject claim — only CloudTrail revealed the real cause

Part 27 — Command and console cheat sheet

27.1 Terraform

```
terraform fmt -recursive          # format all files
terraform fmt -check -recursive   # fail if unformatted (what CI runs)
```

```
terraform validate               # syntax and internal consistency
terraform plan                   # compute the diff; changes nothing
terraform plan -out=FILE         # save the exact plan
terraform apply FILE             # apply exactly that plan, no re-evaluation
terraform state list             # every resource under management
terraform state show ADDRESS     # all attributes of one resource
terraform import ADDRESS ID      # adopt an existing resource
terraform force-unlock LOCK_ID   # last resort only
```

Windows note: PowerShell mangles Terraform flags. Use the stop-parsing token — terraform --% plan -no-color — but it cannot be piped or redirected, so run it bare or use bash.

27.2 Step Functions and Glue

```
aws stepfunctions list-executions --state-machine-arn ARN --max-results 5
aws stepfunctions describe-execution --execution-arn ARN
aws stepfunctions get-execution-history --execution-arn ARN --max-results 50

aws glue get-job --job-name NAME             # definition, including connections
aws glue get-job-runs --job-name NAME --max-results 5
aws glue start-crawler --name NAME
aws glue get-crawler --name NAME             # state and last run
```

27.3 S3, Secrets, SNS, Redshift

```
aws s3 ls s3://BUCKET/silver/ --recursive --summarize | tail -3
aws s3api list-object-versions --bucket BUCKET --prefix KEY

aws secretsmanager list-secrets --query 'SecretList[].Name'
aws secretsmanager describe-secret --secret-id NAME     # metadata, not the value

aws sns list-subscriptions-by-topic --topic-arn ARN     # confirmed vs pending
aws sns publish --topic-arn ARN --subject S --message M # prove delivery

aws redshift-data execute-statement --workgroup-name WG --database DB \
    --secret-arn ARN --sql 'SELECT COUNT(*) FROM fact_trips'
aws redshift-data describe-statement --id ID
aws redshift-data get-statement-result --id ID
```

27.4 CloudTrail — the troubleshooting tool

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRoleWithWebIdentity \
  --max-results 10

aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,AttributeValue=SESSION_NAME
```

Read userIdentity.principalId on a FAILED event. It shows the subject actually presented, which the caller's error message never does. This is what solved the OIDC incident.

27.5 Git hygiene

```
git check-ignore -v PATH          # prove a file is ignored, and by which rule
git ls-files '*.tfvars'           # must return nothing
git diff --cached --ignore-space-at-eol   # content change vs line-ending change
git add --renormalize .           # re-stage under current .gitattributes
```

Part 28 — One-page refresher

Read this alone the morning of an interview.

The pitch

Two datasets needing opposite treatment — 2.85M disposable trip rows and 265 reference rows where every change must survive — processed by a medallion lake and an SCD Type 2 master store, converged into a Redshift star schema behind QuickSight, orchestrated by Step Functions, deployed entirely by Terraform through OIDC-federated GitHub Actions.

The flow

```
bronze/ --silver-etl--> silver/ (Delta) --gold-etl--> gold/ (Delta) --> Athena
                              |
zone CSV --golden-zone-etl--> RDS golden_zones (SCD2)
                              |
                    warehouse-export-etl
                              |
                     warehouse/ (plain Parquet)
                              |  COPY (13 stmts via Data API)
                     Redshift star schema
                              |  SPICE
                     QuickSight (5 sheets, 28 visuals)
```

The numbers

2,851,125	Silver / fact / SPICE	265	zones
33	dim_date	7	dim_payment
18/18	warehouse checks	33/33	days reconciled
17	SFN states (6 stages)	13	load statements
6	Glue jobs (4 orchestrated)	11	Terraform modules
3	crawlers	4	CloudWatch alarms

Say these precisely

- sync-pipeline-runs and delta-demo are NOT Step Functions stages — 4 of 6 Glue jobs are orchestrated
- warehouse/ is NOT crawled — nothing queries it through the catalog
- RDS has no public address, but its subnets route to an IGW — protection is addressing and security groups
- Athena is NOT the dashboard source — QuickSight reads Redshift
- Alarms and EventBridge are INDEPENDENT paths to SNS
- Locking is S3 conditional writes — NO DynamoDB table
- QuickSight is MANUAL — the one plane not reproducible from code
- The Redshift load is NOT atomic — TRUNCATE commits implicitly

The three stories

20. **dim_zone returned 264, not 265** — the SCD2 version-pointer join silently dropped every updated zone. Fixed by resolving on the business key. A count that is almost right is worse than one obviously broken.
21. **OIDC failed with a correct-looking trust policy** — GitHub was issuing an immutable-ID subject claim. Only CloudTrail showed the identity actually presented.
22. **Five phantom Terraform changes every run** — CRLF versus LF changed a file hash. Without .gitattributes, applying from Windows and Linux alternately would rewrite them forever.

— End of guide —